

Análisis Crítico Comparativo de un Sistema Distribuido Real: Google Spanner

Universidad Técnica de Quevedo
Facultad de Ciencias de la Ingeniería
Carrera de Ingeniería en Sistemas

Asignatura: Sistemas Distribuidos — Unidad 1

Modalidad: Individual — Sumativa

Repositorio GitHub:

[https://github.com/DeJere/Aplicaciones-
distribuida-Alvarez-P-rraga-
/tree/9e9b13b0a7386ac473ee15d51757ba78
35ea92af/laxTex%20dis%C3%B1o%20indi
vidual](https://github.com/DeJere/Aplicaciones-distribuida-Alvarez-P-rraga-/tree/9e9b13b0a7386ac473ee15d51757ba7835ea92af/laxTex%20dis%C3%B1o%20individual)

Resumen

Google Spanner es un sistema de gestión de bases de datos distribuido globalmente, desarrollado por Google LLC. Representa uno de los hitos más significativos en la historia de los sistemas distribuidos al lograr semántica ACID completa con transacciones externamente consistentes a escala planetaria. Este informe aplica los conceptos centrales de la Unidad 1 de Sistemas Distribuidos —transparencias de distribución, teorema CAP, modelos de comunicación y tolerancia a fallos— al análisis crítico de la arquitectura de Google Spanner, comparándolo detalladamente con alternativas de la industria como Apache Cassandra y MongoDB bajo la norma estándar IEEE.

Índice

1. Introducción	3
2. Descripción General e Infraestructura Jerárquica	3
2.1. Origen y Evolución: El paso de Bigtable a Spanner.....	3
2.2. Componentes de la Arquitectura Interna.....	3
3. Transparencias de Distribución en Spanner	3
4. Análisis Detallado del Modelo CAP	4
4.1. Clasificación CP frente al Comportamiento Práctico.....	4
4.2. Tabla Comparativa de Sistemas Distribuidos.....	4
5. Modelos de Comunicación y el Ecosistema TrueTime	5
5.1. Comunicación Basada en gRPC y Consenso Paxos.....	5
5.2. TrueTime y el Mecanismo de Commit-Wait.....	5
6. Transacciones Distribuidas: 2PC sobre Paxos	5
7. Tolerancia a Fallos y Mecanismos de Resiliencia	6
8. Conclusiones y Crítica del Sistema	6
Referencias Bibliográficas	7

1. Introducción

Los sistemas de bases de datos distribuidos han evolucionado aceleradamente en las últimas dos décadas, impulsados por la necesidad de escalar a millones de usuarios simultáneos, mantener alta disponibilidad ante fallos de infraestructura y garantizar la coherencia de los datos en entornos geográficamente dispersos. Durante mucho tiempo, la industria asumió que existía una disyuntiva irreconciliable entre consistencia y disponibilidad, formalizada teóricamente por el teorema CAP [2].

Google Spanner irrumpe en este panorama como un sistema que desafía las interpretaciones convencionales de dicho teorema. Al aprovechar una infraestructura coordinada de relojes atómicos y GPS distribuidos —denominada *TrueTime API*— Spanner logra ofrecer transacciones globalmente consistentes con tiempos de latencia competitivos, redefiniendo lo que es posible en sistemas distribuidos de producción a escala global [1]. El objetivo de este informe es desglosar de manera analítica dicho comportamiento utilizando el marco conceptual de la Unidad 1.

2. Descripción General e Infraestructura Jerárquica

2.1. Origen y Evolución: El paso de Bigtable a Spanner

Antes de la creación de Spanner, Google dependía de infraestructuras como Bigtable y Megastore [1]. Si bien Bigtable proporcionaba una escalabilidad horizontal masiva para estructuras Clave-Valor, carecía de soporte para transacciones ACID complejas que involucraran múltiples filas y esquemas relacionales estrictos. Por otro lado, Megastore introdujo transacciones mediante el uso de Paxos, pero sufría de un rendimiento de escritura muy pobre debido a la latencia de la red WAN. Spanner fue diseñado para consolidar lo mejor de ambos mundos: la escalabilidad NoSQL de Bigtable y las garantías transaccionales estrictas de los sistemas relacionales tradicionales.

2.2. Componentes de la Arquitectura Interna

La organización física y lógica de Spanner se estructura de forma estrictamente jerárquica para mitigar los cuellos de botella geográficos:

- **Universe:** Representa una instalación completa de Spanner. Generalmente, Google opera muy pocos *universes* diferenciados (producción, desarrollo y pruebas).
- **Zones:** Son las unidades operativas y de aislamiento administrativo dentro de un *universe*. Equivalen a un centro de datos o a un conjunto de ellos en una misma región geográfica. Las zonas son también la unidad de replicación física.
- **Universe Master y Placement Driver:** El *Universe Master* es una consola central que monitorea el estado de todas las zonas, mientras que el *Placement Driver* maneja de manera automatizada la migración de datos entre zonas según las políticas de latencia y carga.
- **Spanserver:** Servidores físicos encargados de gestionar el almacenamiento y atender las peticiones de los clientes. Cada *Spanserver* maneja entre cien y miles de unidades estructurales llamadas *tablets*.

3. Transparencias de Distribución en Spanner

El modelo de transparencias define en qué medida los detalles intrínsecos de la distribución física son perceptibles para el usuario [3]. Spanner aborda estas dimensiones de la siguiente manera:

- **Transparencia de Acceso:** Spanner abstrae por completo los mecanismos locales de almacenamiento mediante una interfaz SQL estándar (ANSI SQL 2011). Las aplicaciones realizan consultas de forma idéntica sin importar si los datos residen en la memoria local del clúster o en un nodo remoto.
- **Transparencia de Ubicación:** Los datos se organizan en conjuntos de filas denominados *directorios*, los cuales se fragmentan dinámicamente en *splits*. Spanner mueve estos *splits* entre diferentes servidores o zonas geográficas sin que el cliente deba modificar sus cadenas de conexión o conocer la dirección IP del nodo destino.
- **Transparencia de Replicación:** El cliente interactúa con la base de datos como si existiera una única copia. En el fondo, el sistema gestiona múltiples réplicas mediante grupos Paxos redundantes distribuidos de manera global.
- **Transparencia de Concurrencia:** El aislamiento transaccional más fuerte (serializabilidad externa) asegura que múltiples operaciones concurrentes ejecutadas en diferentes continentes se comporten exactamente como si ocurrieran secuencialmente.
- **Transparencia de Fallos:** Si un nodo interrumpe su servicio, los algoritmos de consenso Paxos eligen un nuevo líder de manera automática. El cliente experimenta únicamente una breve fluctuación en la latencia, pero no una pérdida de conectividad o de persistencia de datos.

4. Análisis Detallado del Modelo CAP

4.1. Clasificación CP frente al Comportamiento Práctico

El teorema CAP postula que en presencia de una partición de red (P), un sistema debe elegir entre Consistencia (C) o Disponibilidad (A) [5]. Teóricamente, Google Spanner se posiciona como un sistema **CP**. Ante un aislamiento drástico que impida alcanzar el quórum de la mayoría de las réplicas Paxos, Spanner prioriza la consistencia y rechaza de forma estricta las peticiones de escritura.

Sin embargo, Google opera sobre una infraestructura de red de fibra óptica dedicada y redundante que minimiza la probabilidad de fallos de red (P). Esto permite a Spanner ofrecer un acuerdo de nivel de servicio (SLA) del 99,999 % (los cinco nueves"), operando en condiciones normales con las ventajas de disponibilidad típicas de un sistema AP, pero sin comprometer jamás la consistencia estricta [2].

4.2. Tabla Comparativa de Sistemas Distribuidos

Característica	Google Spanner	Apache Cassandra	MongoDB
Clasificación CAP	CP (Efectivo CA)	AP (Configurable)	CP (Por defecto)
Consistencia Local	Serializabilidad externa	Eventual / Ajustable	Snapshot Isolation
Sincronización	TrueTime (Hardware)	NTP (Software)	Reloj del Sistema
Manejo de Partición	Bloqueo de escrituras	Escritura con divergencia	Re-elección de Primario
Escalabilidad	Horizontal Global	Horizontal en Anillo	Horizontal Sharding

Tabla 1: Comparativa del modelo CAP entre sistemas distribuidos

5. Modelos de Comunicación y el Ecosistema TrueTime

5.1. Comunicación Basada en gRPC y Consenso Paxos

La capa externa de Spanner utiliza el framework de alto rendimiento **gRPC**, el cual aprovecha las ventajas de HTTP/2 como la multiplexación de flujos binarios sobre sockets TCP permanentes [6]. Internamente, la sincronización del estado de las réplicas depende del protocolo de consenso ****Paxos**** [4]. Cada *tablet* de Spanner está asignada a un grupo Paxos. Un líder de Paxos centraliza las peticiones de escritura de ese fragmento específico y propone los cambios a los nodos seguidores.

5.2. TrueTime y el Mecanismo de Commit-Wait

La sincronización temporal tradicional mediante el protocolo NTP introduce un margen de error demasiado alto e impredecible para coordinar ordenamiento causal. Para solucionar esto, la API TrueTime utiliza relojes atómicos de cesio y antenas GPS instalados directamente en cada centro de datos. TrueTime no entrega el tiempo como un punto absoluto, sino como un intervalo cerrado de incertidumbre matemática:

$$t_{abs} \in [TT.now().earliest, TT.now().latest] \quad (1)$$

Donde el valor de incertidumbre máxima se denota por la variable ϵ , la cual se define como:

$$\epsilon = \frac{latest - earliest}{2} \quad (2)$$

En la infraestructura actual de Google, ϵ se mantiene garantizado por debajo de los 7 ms. Para asegurar la consistencia externa, Spanner implementa el algoritmo de ****Commit-Wait**** descrito formalmente a continuación:

Algorithm 1: Protocolo de Commit Wait en Google Spanner

```
Data: Transacción de escritura  $T$ 
Result: Confirmación Física de la Transacción (Commit)
Adquirir bloqueos de dos fases (2PL);
Asignar marca de tiempo de compromiso:  $s = TT.now().latest$ ;
Enviar propuesta de mutación al quórum Paxos;
if La mayoría Paxos confirma la replicación del log then
    // Fase obligatoria de Commit Wait
    while  $TT.now().earliest \leq s$  do
        | Sujetar la transacción (Esperar);
    end
    Aplicar cambios de manera definitiva en el almacenamiento;
    Liberar bloqueos (2PL);
    return Commit Exitoso;
end
return Abortado por falta de quórum;
```

6. Transacciones Distribuidas: 2PC sobre Paxos

Cuando una transacción requiere modificar datos que se encuentran almacenados en múltiples grupos Paxos separados geográficamente, Spanner coordina la operación implementando un protocolo de ****Commit de Dos Fases (2PC)**** distribuido.

En las implementaciones clásicas de sistemas de bases de datos, el protocolo 2PC es altamente vulnerable debido a que el nodo coordinador representa un punto único de fallo (si el coordinador

se cae en medio de la fase de preparación, los participantes quedan bloqueados indefinidamente). Spanner elimina críticamente esta debilidad: tanto los nodos participantes como el coordinador del 2PC no son servidores individuales, sino ****grupos Paxos replicados****. De este modo, si el servidor físico que actúa como coordinador experimenta un fallo de hardware, el grupo Paxos subyacente elige instantáneamente un sustituto que hereda el estado exacto de la transacción, garantizando la resiliencia global de la operación distribuidora.

7. Tolerancia a Fallos y Mecanismos de Resiliencia

El diseño de Spanner proporciona mecanismos de tolerancia a fallos multi-nivel:

- **Fallo a Nivel de Servidor:** Detectado por la ausencia de ráfagas de sincronización (*heartbeats*) en Paxos. Si el nodo caído era un seguidor, el rendimiento no se altera; si era el líder, el quórum elige un nuevo líder en un lapso de 10 a 30 segundos.
- **Fallo a Nivel de Zona o Región:** Gracias a la replicación multi-región activa, si un desastre natural destruye por completo una infraestructura zonal, el tráfico se redirige inmediatamente hacia las zonas geográficas supervivientes que mantengan la mayoría del quórum.
- **Fallo en el Hardware de Tiempo:** Si un receptor GPS o un reloj atómico pierde precisión, la API TrueTime expande automáticamente el valor de la incertidumbre ϵ . Como consecuencia, el lazo de espera del algoritmo de **Commit-Wait** se alarga de forma segura. El sistema experimenta una degradación de rendimiento (mayor latencia), pero la integridad y consistencia de los datos permanecen completamente inalteradas.

8. Conclusiones y Crítica del Sistema

Google Spanner representa la consolidación práctica de la teoría avanzada de los sistemas distribuidos. Su análisis bajo el prisma de la Unidad 1 demuestra que las transparencias de distribución se pueden lograr de manera efectiva a gran escala mediante el uso inteligente y cohesionado de gRPC, Paxos y TrueTime.

La principal limitación del sistema radica en su estrecho acoplamiento con la infraestructura propietaria de Google, lo que induce un riesgo latente de dependencia del proveedor (*vendor lock-in*) y elevados costes operativos. No obstante, al demostrar que las barreras convencionales del teorema CAP se pueden flexibilizar mediante ingeniería de hardware y control riguroso de la incertidumbre del tiempo, Spanner ha redefinido el paradigma del almacenamiento global, inspirando de forma directa la arquitectura de los modernos motores de bases de datos de código abierto englobados en el movimiento *NewSQL*.

Referencias Bibliográficas

Referencias

- [1] J. C. Corbett *et al.*, “Spanner: Google’s Globally-Distributed Database,” in *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI '12)*, Hollywood, CA, USA, 2012, pp. 251–264.
- [2] E. Brewer, “CAP twelve years later: How the rules have changed,” *Computer*, vol. 45, no. 2, pp. 23–29, Feb. 2012.
- [3] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3ra ed. NJ, USA: CreateSpace Independent Publishing Platform, 2017.
- [4] L. Lamport, “The part-time parliament,” *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, May 1998.
- [5] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, Jun. 2002.
- [6] Google Cloud, “Cloud Spanner documentation,” Google Cloud Platform, Tech. Rep., 2024. [En línea]. Disponible en: <https://cloud.google.com/spanner/docs>